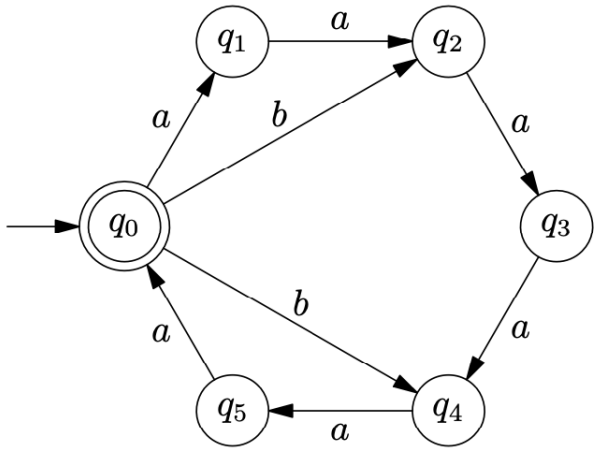
Exercise sheet with solutions

General remarks:

- All tasks are comparable to typical exam tasks. The number of ★'s gives a rough indicator of the task's difficulty.
- You can always ask us for feedback or help during the exercise class.
- You can submit your solution to the task labeled "Homework" via DTU learn (deadline: 6 Feb. at midnight) to receive detailed written feedback, similarly to an exam correction. **We strongly recommend submitting, especially if you are unsure how exactly to formulate your solution.**
- We recommend that you first read all tasks during class. Make sure that you have a rough approach for every task in mind before you start working on the details. Ask for help if a task is unclear such that you do not get stuck at home.
- Solutions will be published a day before the next class; feel free to ask questions during next class after comparing the solution to yours.

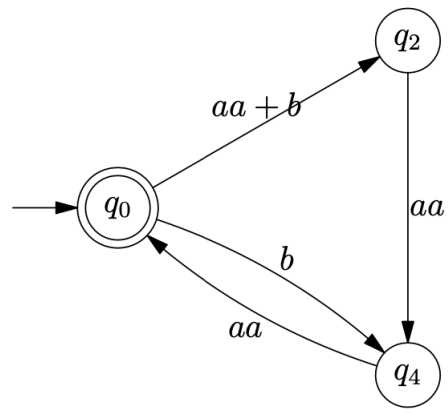
1 From NFAs to Regular Expressions (★)

Construct a regular expression describing the language of the following NFA:

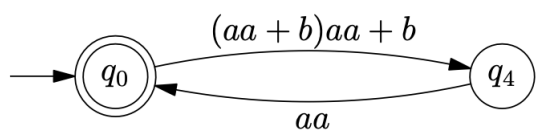


Solution

We first eliminate the states q_1, q_3, and q_5. The resulting automaton is



After that, we remove q_2 and obtain the automaton



We can then derive the regular expression

$$(((aa + b)aa + b)aa)^*.$$

2 True or False? (★★)

Determine for each of the statements below, whether it is true or false. Briefly justify your answer.

(a) There is a regular language L_1 such that L_1 is accepted by an NFA with 10 states, but not accepted by any DFA with 100 states.

(b) There is a regular language L_2 such that L_2 is accepted by an NFA with 10 states, but not accepted by any DFA with 1217 states.

(c) For every regular expression E over Σ, there exists a regular expression E' over Σ such that $L(E') = \Sigma^* \setminus L(E)$.

Solution

(a) Correct. As noted in the lecture, there is an NFA with $n + 2$ states that accepts the language $(a + b)^* a(a + b)^n$. However, every DFA accepting the same languages requires at least 2^n states.

(b) Wrong. If there is an NFA with 10 states that accepts a language L, then there is a DFA accepting L that is obtained via the subset/powerset construction. This DFA has $2^{10} = 1024 < 1217$ states.

(c) Correct. We can transform E' into a DFA A accepting $L(E')$ via Thompson's construction, removal of ε-transitions, and powerset construction. We can then construct an automaton B that accepts $\Sigma^* \setminus L(A)$ by swapping final and non-final states. After that, we can construct a regular expression E' with $L(E') = L(B) = \Sigma^* \setminus L(E)$ using the state elimination algorithm.

3 Homework: Read the Formal Manual (★★)

A machine has two valves, which can be opened for one second using the commands a and b. The machine's manufacturer provides no warranty if one does not adhere to the opening sequences specified in the manual by the NFA in Figure 1. To be more precise: After the machine has been switched on, we are allowed to open the valves according to a permitted opening sequence and then switch the machine off again. For example, it is *not* permitted to switch the machine on and off again immediately, as there is no safe opening sequence according to the manual, i.e. the word ε is not accepted by the NFA in Figure 1.

(a) Is the opening seqeunce $aaabaabaa$ safe or unsafe?

(b) Construct a DFA and a regular expression that describes all unsafe opening sequences, i.e. precisely those words that are *not* accepted by the above NFA. Explain which constructions you use.

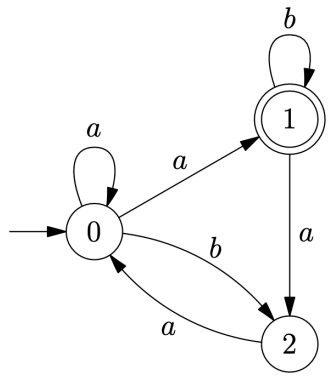


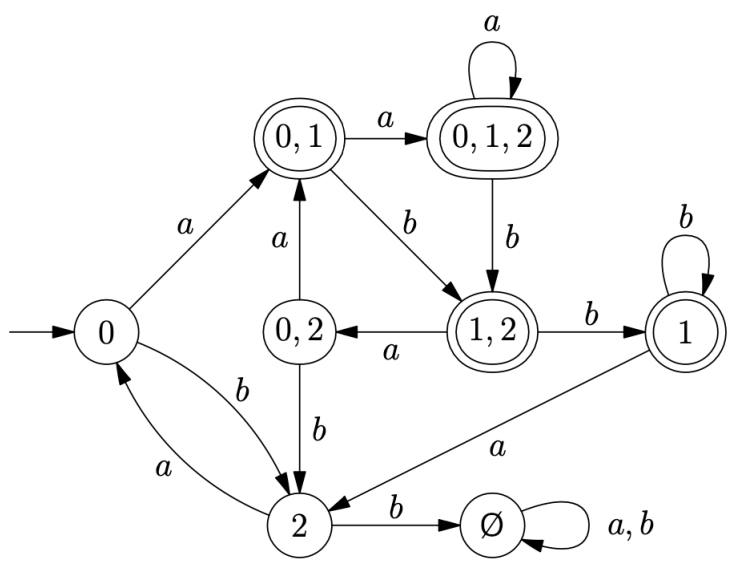
Figure 1: Specification of safe opening sequences.

Solution

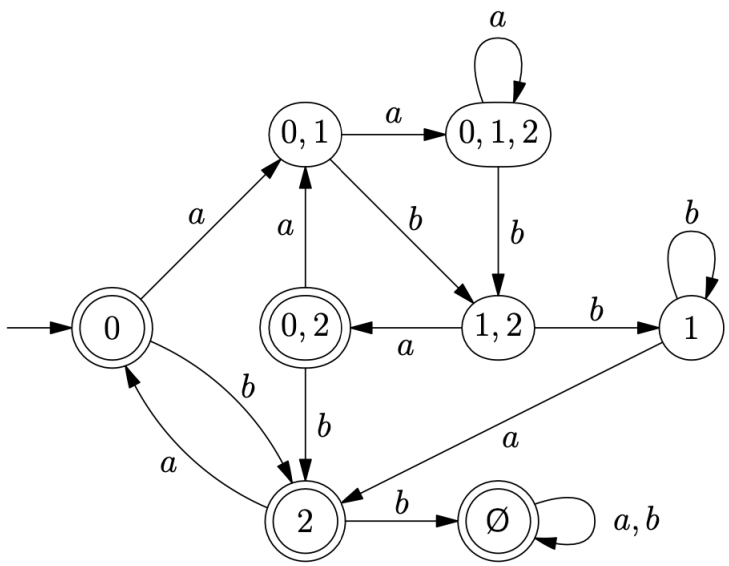
(a). Yes, because there is an accepting run of the DFA in Figure 1:

$$0 \xrightarrow{a} 0 \xrightarrow{a} 0 \xrightarrow{a} 0 \xrightarrow{b} 2 \xrightarrow{a} 0 \xrightarrow{a} 0 \xrightarrow{b} 2 \xrightarrow{a} 0 \xrightarrow{a} 1$$

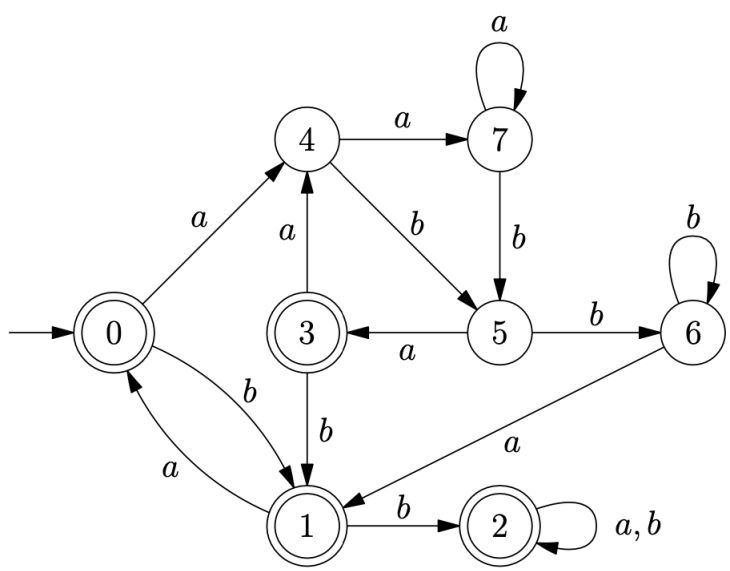
(b). We first apply the powerset construction to obtain a DFA accepting the same language as the NFA in Figure 1:



By swapping final and non-final states, we then obtain a DFA accepting the negation of the above DFA. This new DFA accepts exactly the unsafe opening sequences:



To improve readability, we rename the states before we proceed:



We can now apply the state elimination algorithm to construct a regular expression accepting the same language. The precise shape of the expression highly depends on the order in which states are eliminated.

One possible regular expression is

$$(a^+ba + (b + a^+bb^+a)a)^*(\varepsilon + (b + a^+bb^+a)(\varepsilon + b(a + b)^*)).$$

4 Commutating Words (★★)

Let $v, w \in \Sigma^*$ such that $vw = wv$. Prove that there exists a word $u \in \Sigma^*$ and natural numbers $i, j \in \mathbb{Z}_{\geq 0}$ such that $v = u^i$ and $w = u^j$.

Hints: Proceed by induction over the length of the word $|vw|$. If you do not remember induction proofs from discrete math, ask for help. Moreover, recall that u^i is given by $u^0 = \varepsilon$ and $u^{n+1} = uu^n$.

Solution

Proof by induction on $|vw|$.

Induction base: If $|vw| = 0$, then $v = w = \varepsilon$. Then, for $u = \varepsilon$, we have $v = u^0 = w$.

Induction hypothesis (I.H.): Assume for an arbitrary, but fixed, length $n >= 0$ that, for all words x, y with $|xy| \leq n$ and $xy = yx$, there exists a word $u \in \Sigma^*$ and natural numbers $i, j \in \mathbb{Z}_{\geq 0}$ such that $x = u^i$ and $y = u^j$.

Induction step: Assume $|vw| > 0$. Then two cases are possible:

1. If $|v| = |w|$, then $v = w$ and thus the claim holds for $u = v = w$ and $i = j = 1$.
2. If $|v| > |w|$, then there is a word $x \in \Sigma^+ = \Sigma\Sigma^*$ such that $vx = w$ and $xv = w$. Since $|vx| < |vw|$, we know by I.H. that there exists a word $u \in \Sigma^*$ and $i', j' \geq 0$ such that $x = u^i$ and $v = u^j$. Then the claim holds for u, $i = i'$ and $j = i' + j'$, because $w = u^{i'+j'}$.
3. If $|v| < |w|$, the proof is completely analogous to the case $|w| < |v|$.

5 Walking Through a Museum (★★★)

Figure 2 depicts the plan of a museum with three rooms A, B, and C and connections between those rooms. A valid path through the museum is a non-empty word over $\Sigma = \{A, B, C\}$ that starts and ends at the museum's entry (A). Moreover, paths do not allow "staying" in a room. For example, $ABCABABCA$ is a valid path through the museum, whereas $ABBA$ or ε are not.

(a) Provide a regular expression over the alphabet $\Sigma = \{A, B, C\}$ that describes all valid paths through the museum.

(b) Construct a deterministic finite automaton that accepts all valid paths through the museum.

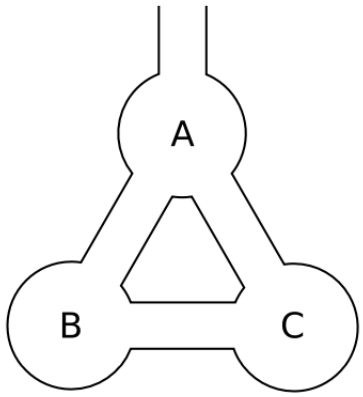


Figure 2: Plan of a museum with rooms A, B, and C.

Solution

(a). Intuitively, we start and end in room A and, in between, move back and forth between the rooms B and C. The back and forth moves are described by the following regular expression:

$$H = (BC)^+ + (BC)^*B + (CB)^* + (CB)^*C,$$

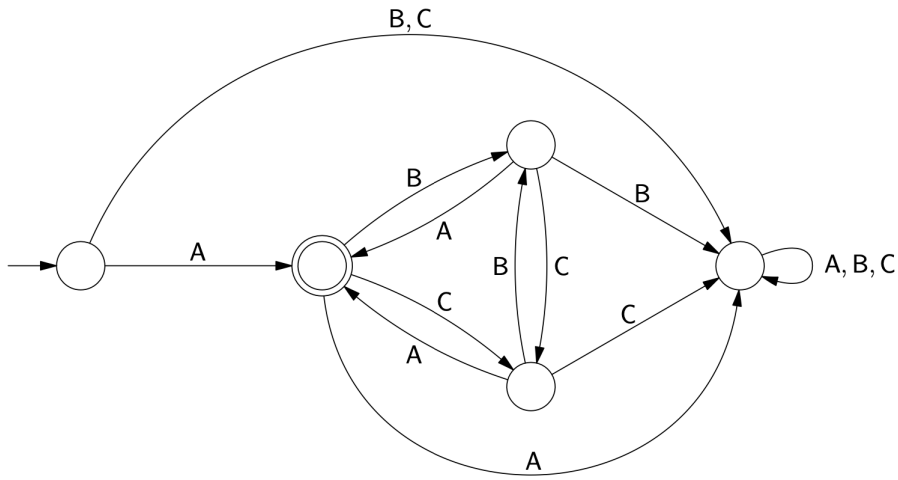
where $L^+ = LL^*$. The four summands can be understood as follows:

1. We start in B and end in C.
2. We start in B and end in B.
3. We start in C and end in B.
4. We start in C and end in C.

Now, it suffices to start and end in A, i.e. the overall regular expression is

$$(AH)^*A \quad = \quad \left(A\left((BC)^+ + (BC)^*B + (CB)^+ + (CB)^*C\right)\right)^* A.$$

(b). One could, of course, transform the above regular expression into a DFA. However, one can just as well construct a DFA directly:



6 Modulo Counting (★★★)

Construct a regular expression over the alphabet $\Sigma = \{a, b\}$ whose language consists of all words, which contain an even number of a's *if and only if* they also contain an even number of b's.

Solution

We first construct a simple DFA with 4 states that counts the number of a's and b's modulo 2.

Assume the states of this DFA are $q_{a0,b0}$, $q_{a0,b1}$, $q_{a1,b0}$, $q_{a1,b1}$, where $a1$ indicates that we have read an odd number of a's and $b0$ indicates that we have read an even number of b's. Hence, the states $q_{a0,b0}$ (even number of a's and b's) and $q_{a1,b1}$ (odd number of a's and b's) are final states.

For these two final states, we obtain the following regular expressions via the state elimination algorithm from the lecture:

$$\begin{aligned} L_1 &= (aa + bb + (ab + ba)(aa + bb)^*(ab + ba))^* \\ L_2 &= (aa + bb)^*(ab + ba)\,(aa + bb + (ab + ba)(aa + bb)^*(ab + ba))^* \end{aligned}$$

A suitable regular expression is then given by $L_1 + L_2$.

We can even obtain a more elegant regular expression by observing that $L_2 = (aa + bb)^*(ab + ba)L_1$. Then a regular expression for the whole language is given by

$$(\varepsilon + (aa + bb)^*(ab + ba))\,(aa + bb + (ab + ba)(aa + bb)^*(ab + ba))^* \,.$$